Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Интеллектуальный анализ данных»

Тема: «Автоэнкодеры»

Выполнил:
Студент 4 курса
Группы ИИ-26
Яшков П. А.

Проверил:
Никитин С. К.

Брест 2026

Цель работы:

Научиться строить и анализировать низкоразмерные представления при помощи автоэнкодеров, PCA и t-SNE.

Общее задание

1. На варианте WDBC обучить автоэнкодер с bottleneck 2 и 3 нейрона.
2. Визуализировать проекции по классам с помощью matplotlib.
3. Применить t-SNE с 2 и 3 компонентами и исследовать параметр perplexity.
4. Применить PCA из ЛР №1 на 2 и 3 компонентах для сравнения.
5. Подготовить отчёт, графики и исходный код.

Сведения об эксперименте

Автор: Яшков П. А., группа ИИ-26, 4 курс. Вариант: 11. Все расчёты воспроизводимы: random_state/seed = 42. Исходники размещены в папке src/, результаты — src/results/.

Данные и подготовка

WDBC (Wisconsin Diagnostic Breast Cancer) содержит 569 наблюдений и 30 числовых диагностических признаков. Во втором столбце исходного формата находится метка диагноза (M — malignant, B — benign); идентификатор в признаки не включается. Данные стандартизируются, затем масштабируются в [0,1] для автоэнкодера с Sigmoid-декодером.

Архитектура и методы

Обучены два полносвязных автоэнкодера 30 → 32 → 2/3 → 32 → 30. В скрытых слоях используется ReLU, в выходном — Sigmoid; оптимизатор Adam, функция потерь MSE. Для проверки строятся 2D и 3D проекции PCA, а также визуализации t-SNE при perplexity 20, 40 и 60. Для 3D t-SNE взята воспроизводимая подвыборка 350 объектов, поскольку точный t-SNE значительно более ресурсоёмок.

Численные результаты

Кодировщик	MSE, начало	MSE, конец	Эпох
2	0.10756	0.00879	70
3	0.09924	0.00827	70

t-SNE	Perplexity	KL divergence
2D	20	1.0416
2D	40	0.8212
2D	60	0.6900
3D	20	0.6442
3D	40	0.4637
3D	60	0.3381

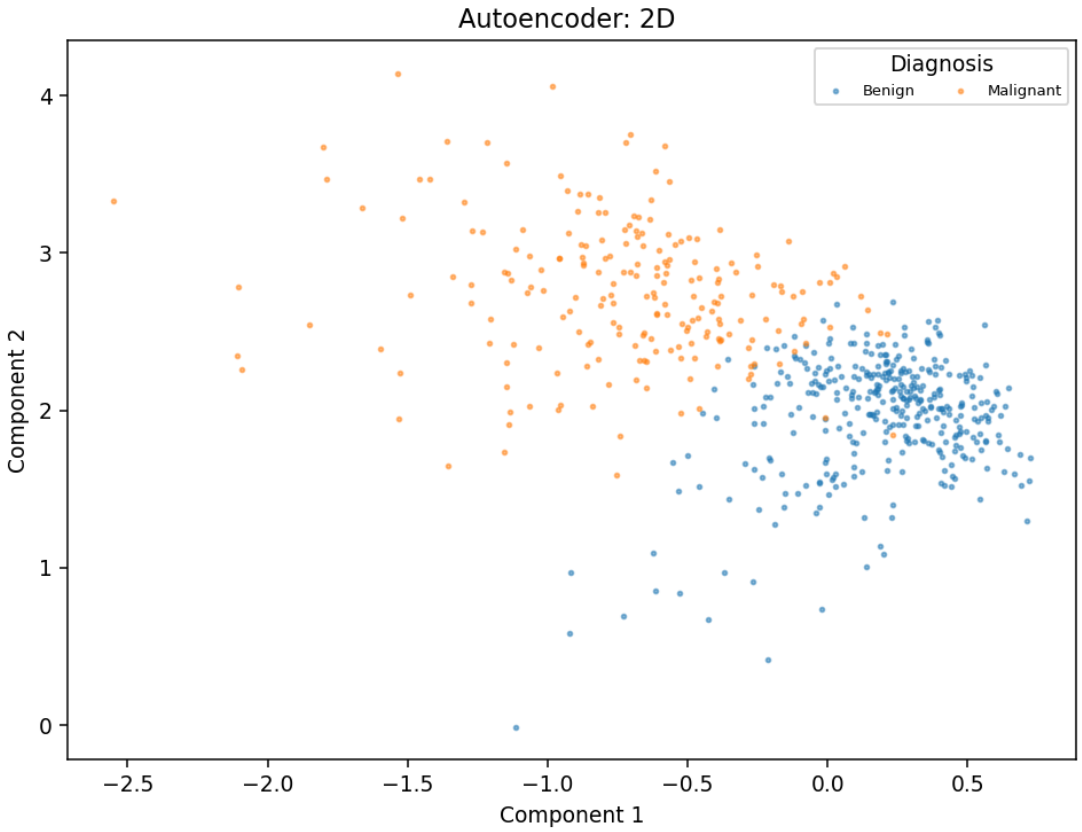


Рисунок 1 — Проекции автоэнкодера: 2D.

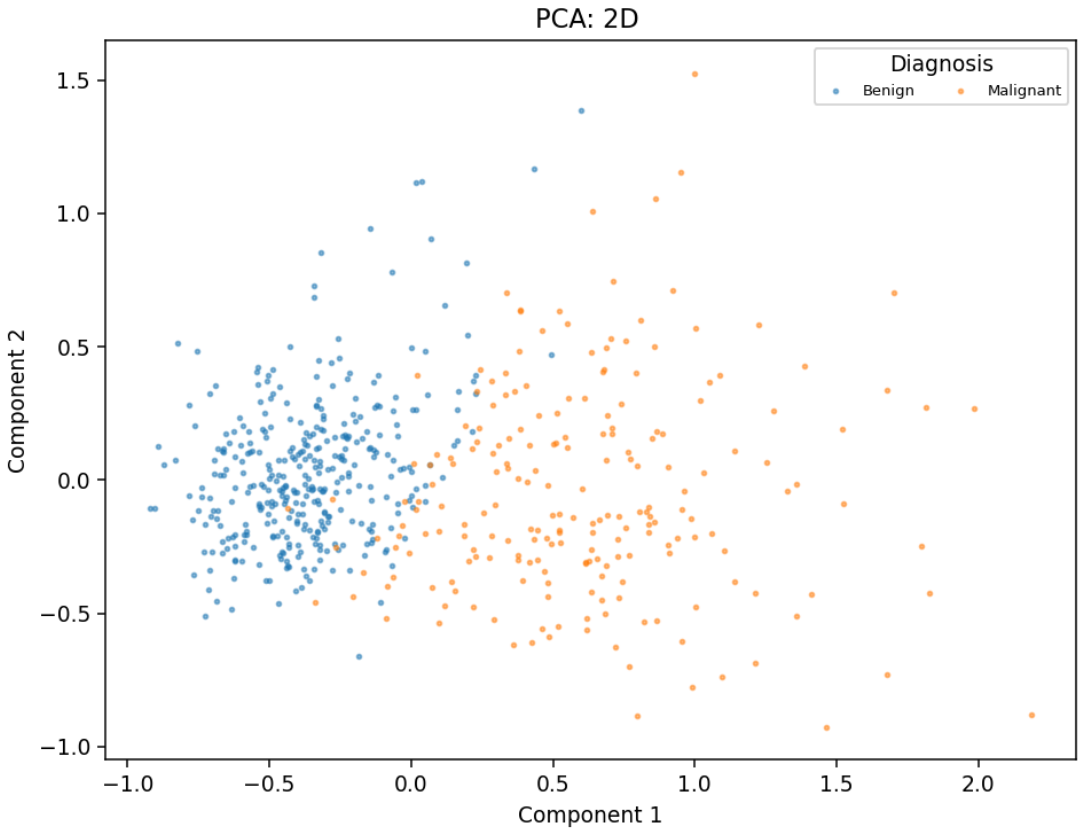


Рисунок 2 — PCA: 2D.

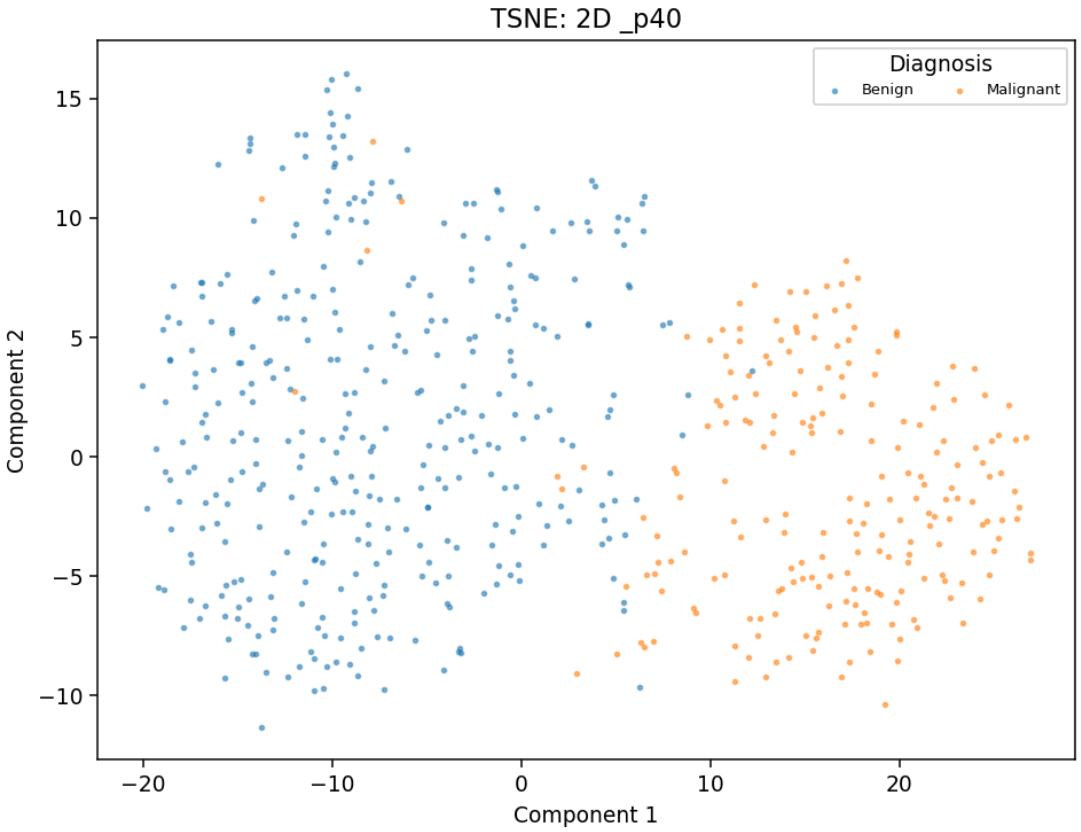


Рисунок 3 — t-SNE: 2D, perplexity 40.

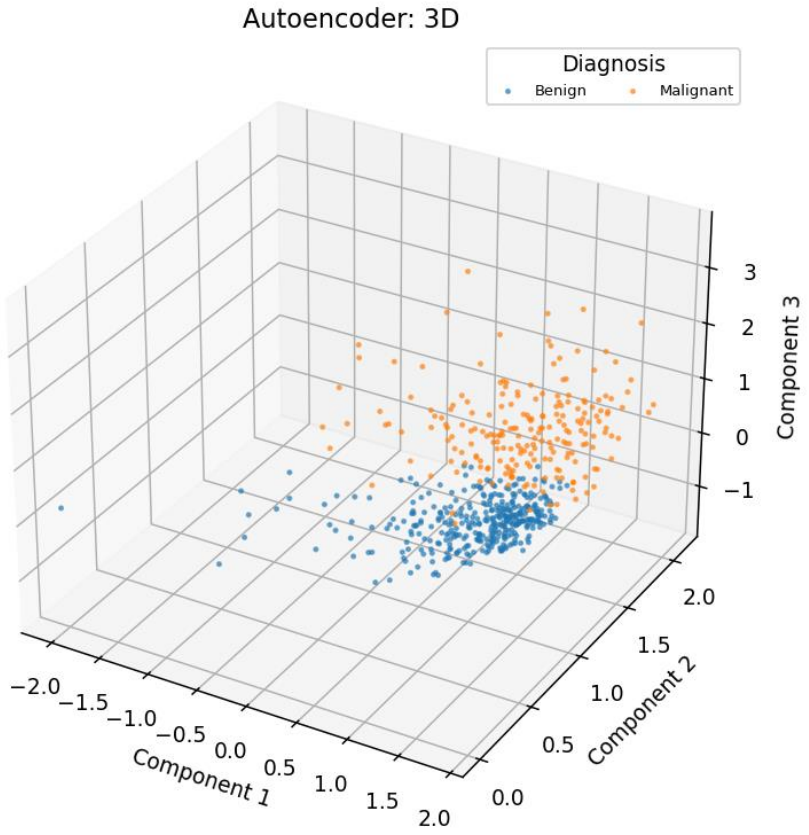


Рисунок 4 — Автоэнкодер: 3D.

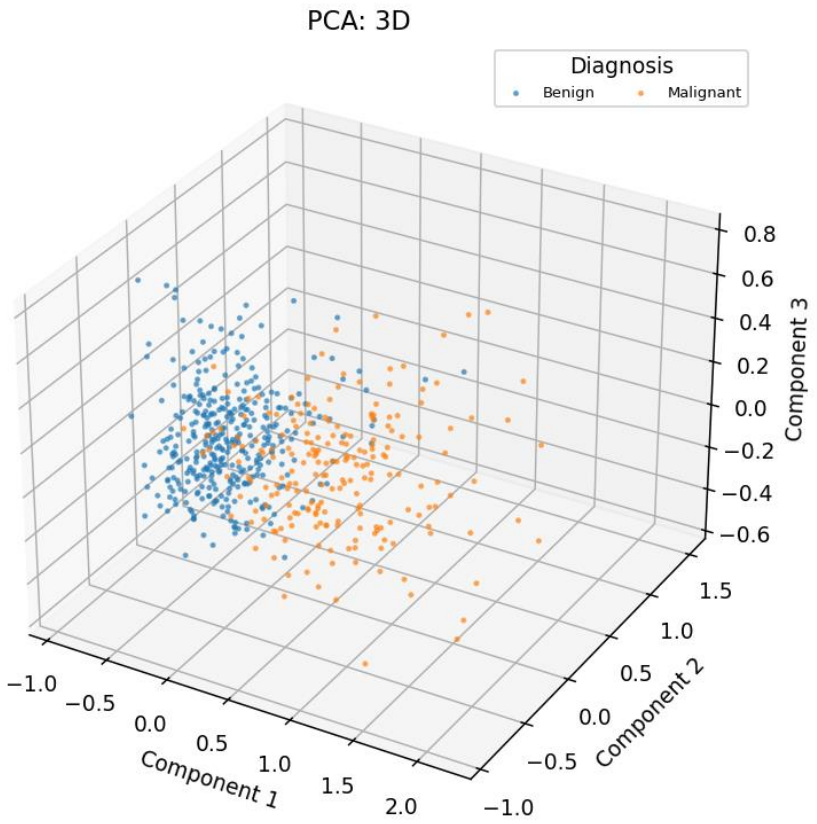


Рисунок 5 — PCA: 3D.

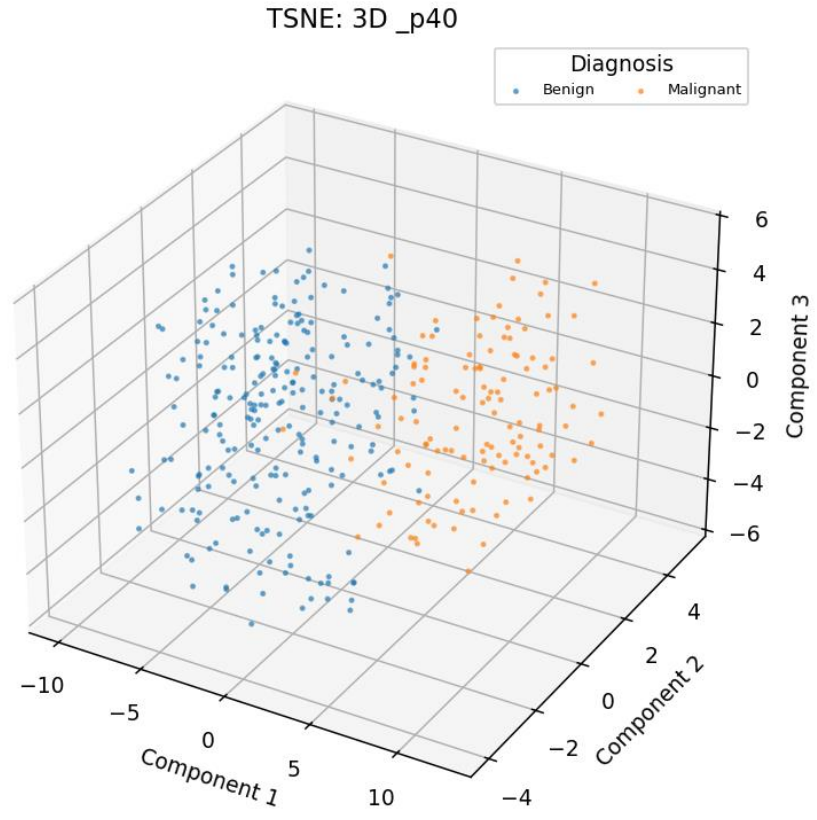


Рисунок 6 — t-SNE: 3D, perplexity 40.

Эксперимент с параметром perplexity

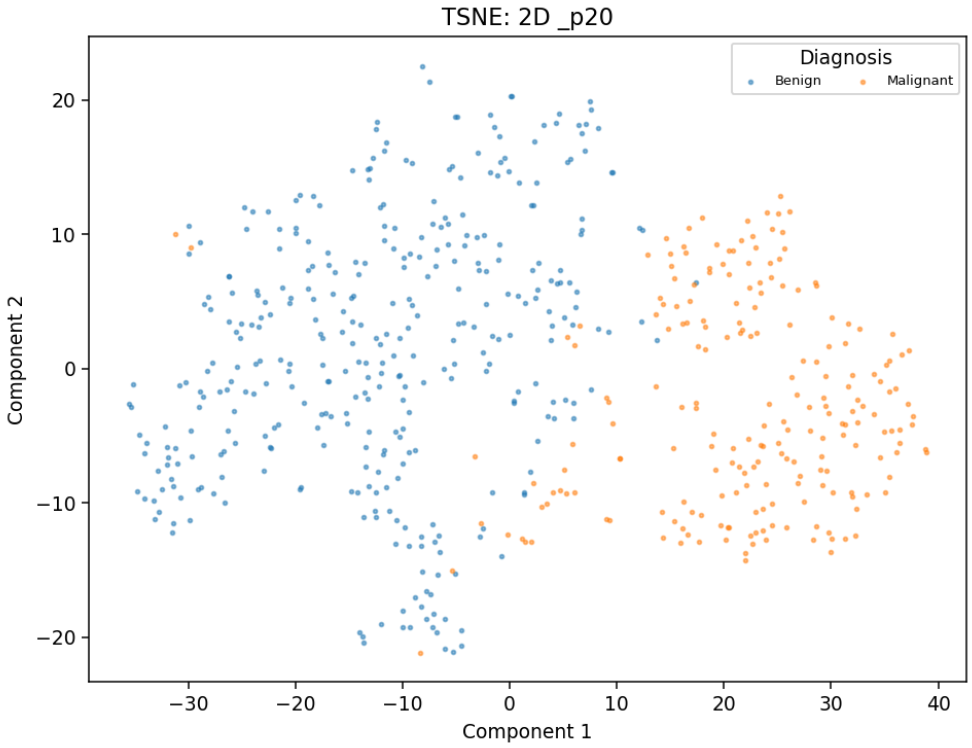


Рисунок — t-SNE 2D, perplexity 20.

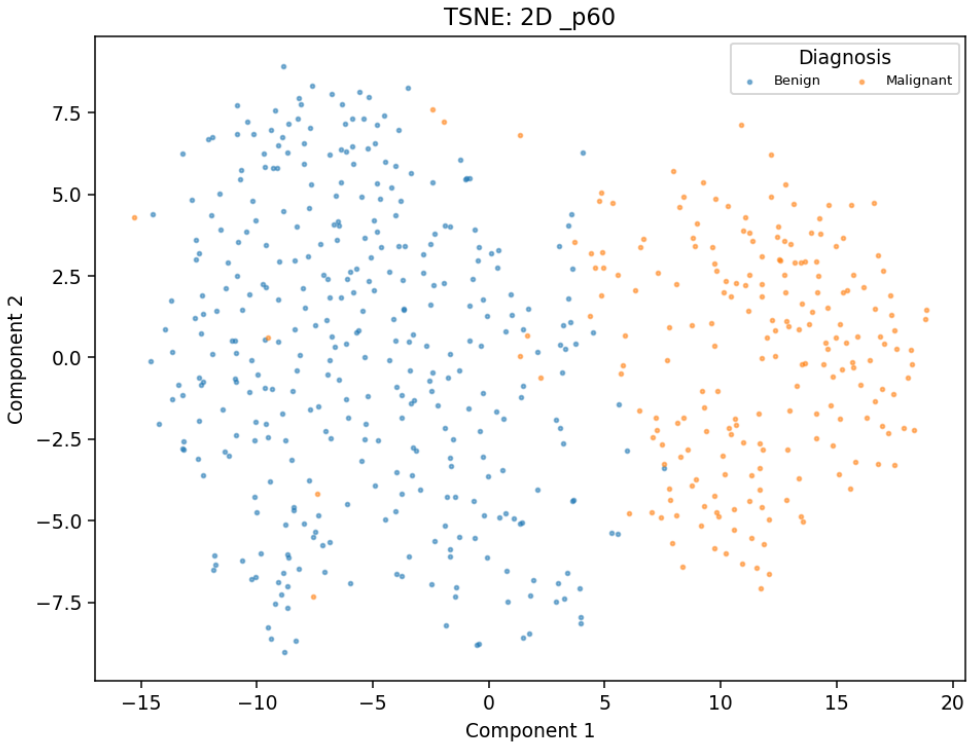


Рисунок — t-SNE 2D, perplexity 60.

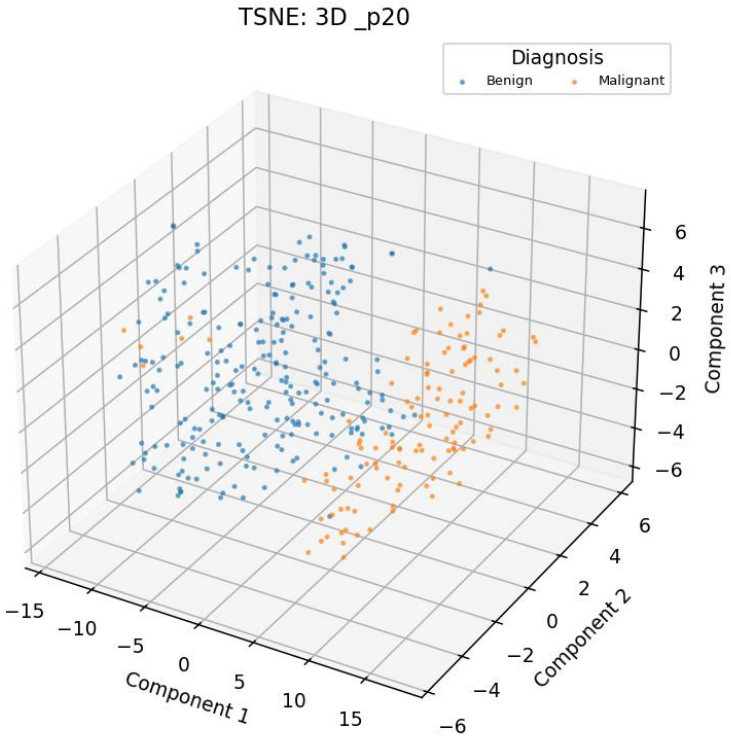


Рисунок — t-SNE 3D, perplexity 20.

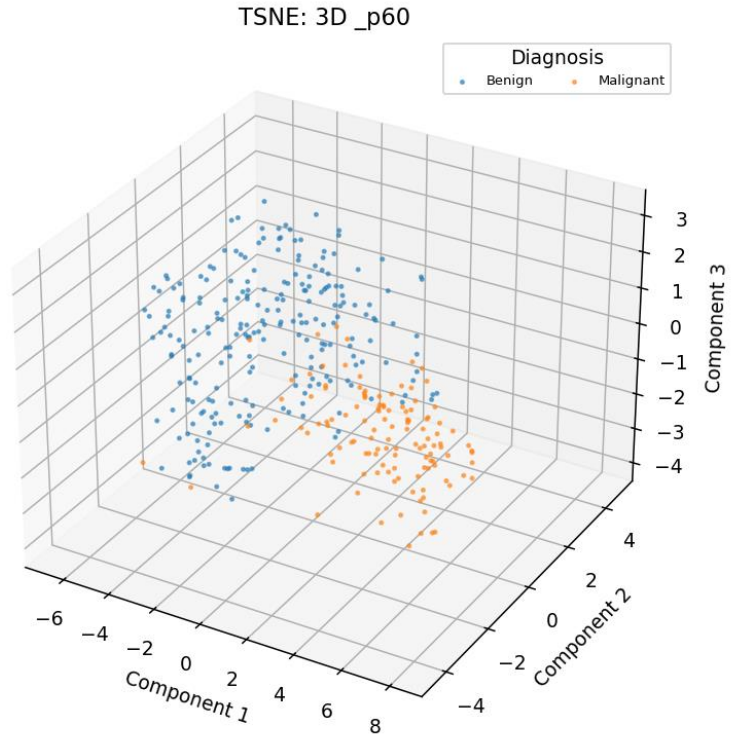


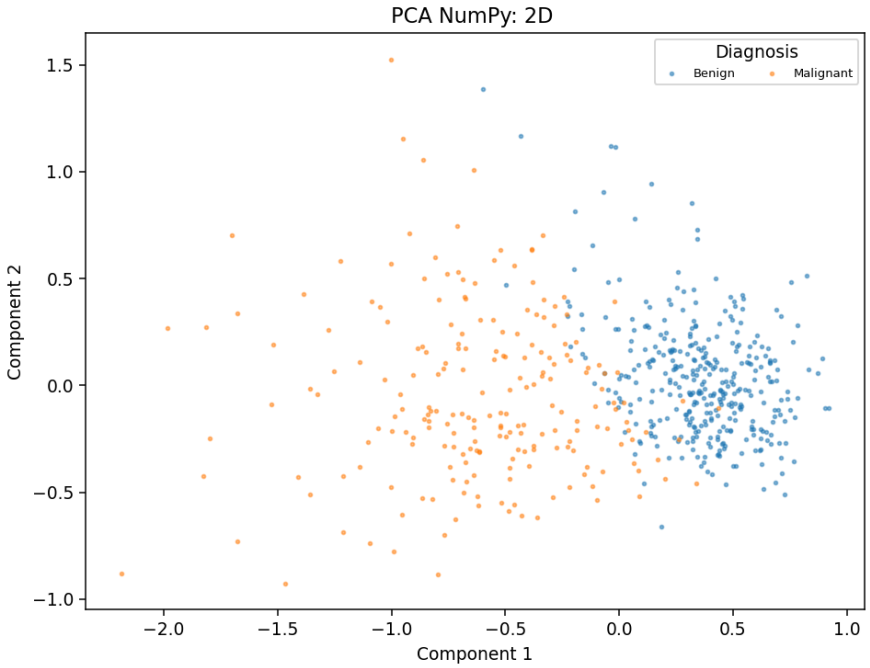
Рисунок — t-SNE 3D, perplexity 60.

Дополнительная проверка PCA методом NumPy из лабораторной №1

Помимо библиотечной реализации PCA повторён ручной алгоритм из ЛР №1: центрирование, вычисление ковариационной матрицы, собственных значений и векторов с помощью numpy.linalg.eigh, сортировка по убыванию собственных значений и проекция на 2 и 3 компоненты. Для численно

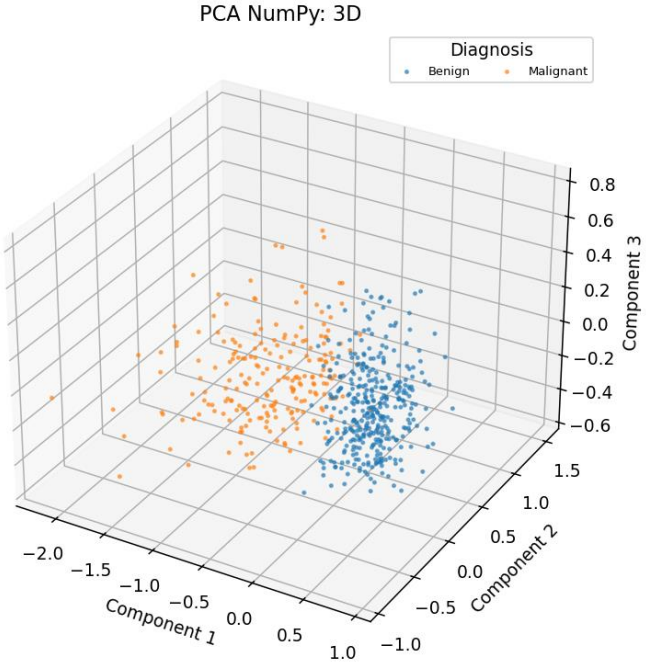
симметричной ковариационной матрицы eigh является более устойчивой разновидностью используемого в ЛР №1 eig. Результаты дополнительно сверены со sklearn.PCA.

2 компоненты: объяснённая дисперсия 70.38%, потеря информативности 29.62%. Максимальное расхождение собственных значений между NumPy и sklearn: 2.00e-07.



Ручной PCA — 2D

3 компоненты: объяснённая дисперсия 77.50%, потеря информативности 22.50%. Максимальное расхождение собственных значений между NumPy и sklearn: 2.00e-07.



Ручной PCA — 3D

Выводы о проделанной работе

1. Автоэнкодер успешно уменьшает число признаков с 30 до 2 или 3 и восстанавливает исходные значения с малой MSE.
2. PCA — линейное преобразование, автоэнкодер — нелинейное, а t-SNE оптимизирует визуальное сохранение локального соседства. Оси t-SNE нельзя интерпретировать как проценты сохранённой дисперсии.
3. Разные значения perplexity дают различную геометрию проекции и величину KL divergence; сравнивать их следует также по читаемости структуры классов.
4. Полученные двумерные и трёхмерные графики позволяют сравнить структуру классов M/B, не используя диагностическую метку при обучении преобразования.

Приложение — исходный код ИАД_2.py

Актуальный исполняемый код файла ИАД_2.py; оригинал находится также в src/.

```
"""ИАД, ЛР №2, вариант 11: WDBC + Autoencoder, t-SNE, PCA."""
from pathlib import Path
import json,random
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import torch
from torch import nn
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
SEED=42
random.seed(SEED);np.random.seed(SEED);torch.manual_seed(SEED);torch.set_num_threads(2)
ROOT=Path(__file__).resolve().parent;OUT=ROOT/'results';OUT.mkdir(exist_ok=True)
df=pd.read_csv(ROOT/'data/wdbc.csv')
# Второй столбец исходного WDBC — диагноз M/B, ID исключается.
y=(df['Diagnosis']=='M').astype(int).values
features=df.drop(columns=['ID','Diagnosis'])
X=StandardScaler().fit_transform(features).astype(np.float32)
# Масштабирование к [0,1] для декодера с Sigmoid.
from sklearn.preprocessing import MinMaxScaler
X=MinMaxScaler().fit_transform(X).astype(np.float32)
class AE(nn.Module):
 def __init__(self,d,latent):

super().__init__();self.enc=nn.Sequential(nn.Linear(d,32),nn.ReLU(),nn.Linear(32,latent));self.dec=nn.Sequential
(nn.Linear(latent,32),nn.ReLU(),nn.Linear(32,d),nn.Sigmoid())
 def forward(self,x):return self.dec(self.enc(x))


def train(latent,epochs=70):
 model=AE(X.shape[1],latent);opt=torch.optim.Adam(model.parameters(),lr=.002)
 loader=torch.utils.data.DataLoader(torch.from_numpy(X),batch_size=256,shuffle=True)
 losses=[]
 for ep in range(epochs):
  total=0
  for z in loader:
   opt.zero_grad();loss=nn.functional.mse_loss(model(z),z);loss.backward();opt.step();total+=float(loss)*len(z)
  losses.append(total/len(X))
 with torch.no_grad():out=model.enc(torch.from_numpy(X)).numpy()
 return model,out,losses


def graph(data,kind,k,suffix='',labels=None):
 if labels is None: labels=y
 fig=plt.figure(figsize=(7.2,5.6));ax=fig.add_subplot(111,projection='3d' if k==3 else None)
 for digit in range(2):
  v=data[labels==digit]
  if k==3:ax.scatter(*v.T,s=3,alpha=.55,label=('Benign' if digit==0 else 'Malignant'))
  else:ax.scatter(v[:,0],v[:,1],s=4,alpha=.5,label=('Benign' if digit==0 else 'Malignant'))
 ax.set(xlabel='Component 1',ylabel='Component 2',title=f'{kind}: {k}D {suffix}')
 if k==3:ax.set_zlabel('Component 3')
 ax.legend(title='Diagnosis',ncol=2,fontsize=7,loc='best')
 fig.tight_layout();fig.savefig(OUT/f'{kind.lower().replace(" ","_")}_{k}d{suffix}.png',dpi=135);plt.close(fig)


metrics={'dataset':'UCI WDBC, diagnosis (second
column)','samples':len(X),'features':X.shape[1],'classes':np.unique(y).tolist(),'autoencoders':{},'pca':{},'tsne
':{},'tsne_sample_size':{'2D':569,'3D':350}}
for k in (2,3):
 model,z,h=train(k);graph(z,'Autoencoder',k)
 torch.save(model.state_dict(),OUT/f'autoencoder_{k}d.pt')
 metrics['autoencoders'][str(k)]={'epochs':len(h),'initial_mse':h[0],'final_mse':h[-1]}
 plt.figure(figsize=(6,3));plt.plot(h);plt.title(f'Autoencoder {k}D — training
MSE');plt.xlabel('Epoch');plt.ylabel('MSE');plt.tight_layout();plt.savefig(OUT/f'ae_training_{k}d.png',dpi=130);
plt.close()
 # PCA from Lab 1: centered covariance and explicit eigenvectors (NumPy).
 centered=X-X.mean(axis=0,keepdims=True)
 covariance=np.cov(centered,rowvar=False)
 eigenvalues,eigenvectors=np.linalg.eigh(covariance)
 order=np.argsort(eigenvalues)[::-1]
 eigenvalues=eigenvalues[order];eigenvectors=eigenvectors[:,order]
 projected_numpy=centered@eigenvectors[:,:k]
```

```
 graph(projected_numpy,'PCA NumPy',k)
 model_pca=PCA(n_components=k,svd_solver='full')
 projected_sklearn=model_pca.fit_transform(X)
 graph(projected_sklearn,'PCA',k)
 diff=float(np.max(np.abs(eigenvalues[:k]-model_pca.explained_variance_)))
 assert diff<1e-6,(k,diff)
 metrics['pca'][str(k)]={'retained_variance_percent':float(100*eigenvalues[:k].sum()/eigenvalues.sum()),
                         'loss_percent':float(100*(1-eigenvalues[:k].sum()/eigenvalues.sum())),
                         'numpy_sklearn_eigenvalue_max_diff':diff}


 # t-SNE сравниваем при нескольких значениях perplexity.
 for p in (20,40,60):
  # Случайная воспроизводимая подвыборка для вычислительно дорогого t-SNE
  chosen=np.random.default_rng(SEED).choice(len(X),size=(350 if k==3 else len(X)),replace=False)

ts=TSNE(n_components=k,perplexity=p,init='pca',learning_rate='auto',random_state=SEED,max_iter=500,method='barne
s_hut' if k==2 else 'exact')
  q=ts.fit_transform(X[chosen])
  graph(q,'TSNE',k,f'_p{p}',labels=y[chosen])
  metrics['tsne'][f'{k}D_perplexity_{p}']={'kl_divergence':float(ts.kl_divergence_)}
(OUT/'metrics.json').write_text(json.dumps(metrics,indent=2,ensure_ascii=False),encoding='utf8')
print(json.dumps(metrics,ensure_ascii=False,indent=2))
```